

Úvod

Cielom projektu je importovať a uložiť veľké množstvo dát z Twitteru vo formáte .jsonl.gz do PostgreSQL databázy. Dátové súbory obsahujú informácie o používateľoch, tweetoch, miestach, hashtagoch, médiách, URL a zmienkach používateľov. Kľúčovou požiadavkou je efektívne spracovanie veľkých datasetov a zabezpečenie správnej integrácie relácií medzi entitami (napr. retweety a citácie).

Postup

Načítanie a parsing súborov:

- Vyhľadanie všetkých .gz súborov v zvolenom adresári.
- Otvorenie súborov pomocou gzip a načítanie riadok po riadku.
 - Využíva sa knižnicaorjson pre rýchle parsovanie
- Extrakcia dát pre jednotlivé entity: User, Place, Tweet, Hashtag, TweetHashtag, TweetURL, TweetUserMention, TweetMedia.

Ukladanie dát do databázy:

- Nezávislé tabuľky (User, Place, Hashtag) sú vložené ako prvé.
- Následne sa vložia tweet objekty (Tweet).
- Závislé tabuľky (TweetHashtag, TweetURL, TweetUserMention, TweetMedia) sa ukladajú paralelne pomocou vlákien (ThreadPoolExecutor) pre zrýchlenie procesu.
- Na záver sa aktualizujú relácie medzi tweetmi (retweeted_status_id, quoted_status_id).
 - využíva sa update pre efektívne aktualizácie

Bezpečnosť a stabilita:

- Použitie bulk_save_objects pre ORM objekty, čím sa eliminujú chyby s NULL hodnotami (SQLAlchemy CompileError).
- Použitie batching-u na spracovanie veľkého počtu záznamov bez preťaženia pamäte spoločne s multiprocessingom.
- Spracovanie chýb pri duplicitných kľúčoch (IntegrityError).

Dáta

Typy entít:

- **User** – informácie o používateľoch (ID, meno, screen_name, popis, URL, overenie, štatistiky).
- **Place** – miesto, odkiaľ tweet pochádza (ID, názov, krajina).
- **Tweet** – samotné tweety, čas, text, jazyk, metriky (retweety, likes).
- **Hashtag** – hashtagy používané v tweetoch.
- **TweetHashtag** – relácia medzi tweetom a hashtagom.
- **TweetURL** – URL odkazy v tweetoch.
- **TweetUserMention** – zmienky používateľov v tweetoch.
- **TweetMedia** – mediálne súbory priradené k tweetom (obrázky, videá).

Spracovaná dáta

Dátové súbory sa postupne parsujú a ukladajú do dočasných štruktúr (slovníky a zoznamy).

Relácie, ktoré závisia na existencii iných entít, sa ukladajú až po vložení hlavného objektu (napr. tweet pred vložením zmienok, médií a hashtagov).

Paralelizácia vloženia závislých tabuliek výrazne znižuje čas spracovania.

ORM metódy (bulk_save_objects) zabezpečujú korektné uloženie aj pri prítomnosti NULL hodnôt a minimalizujú riziko deadlock-ov v databáze.

table_name name	estimated_row_count bigint	total_size text
tweets	6345714	2641 MB
tweet_user_mentions	6805734	697 MB
users	3436581	654 MB
tweet_urls	1422225	271 MB
tweet_hashtags	2326901	166 MB
tweet_media	394535	110 MB
hashtags	218463	25 MB
places	11686	1640 kB

Čas, rýchlosť

Ako prvé som skúšal klasické ukladanie do databázy, kde vloženie jedného súboru trvalo do hodiny. Následne som aplikoval vyššie prvotné vylepšenia, kde sa mi podarilo čas pre jeden súbor dostať pod 10 minút a celkový čas potrebný pre všetky súbory (úprava a vkladanie) bol 3 hodiny. Následne som upravil finálne vkladanie a spracovanie kde som na dostal na celkový čas 50 min, z toho príprava dát 10 min. Vkladanie dát prebiehalo priemernou rýchlosťou 3000 kb/s. Počet chýbajúcich záznamov in_reply_to_status_id:505621.

System

- RAM: 16 GB
- CPU: 8 jadier 2 GHZ
- DISK 512 GB SSD
- OS: windows 10
- Verzia postgres: 16

Zhodnotenie

Podarilo sa spracovať väčší objem dát a uložiť ho. Boli aplikované metódy pre zrýchlenie nahrávania. Skúšal som aj upraviť nastavenie pre databázu ktoré tiež zlepšilo nahrávanie.